

```

/*
* =====
* INVENTORY MANAGEMENT SYSTEM - INDUSTRIAL LEVEL
* Multi-Department: Product | Price | Stock | Maintain | Sales | Admin
* =====
*
* BUGS FIXED:
* 1. Password validation added (min 6 chars, must have letter + digit + symbol)
* 2. Infinite recursion removed - loop-based retry instead of recursive call
* 3. Attempt counter 'ii' made per-function (not shared globally)
* 4. price() - wrong password ke baad price entry nahi hoti ab (return added)
* 5. stock() - same fix applied
* 6. salesteam() - choice==2 (rate change) case ab kaam karta hai
* 7. menu() switch restructured - no more nested if(choice==X) inside case X
* 8. maintain() - wrong ID aur wrong password ke alag messages
* 9. GST - ab har item ka alag GST calculate hota hai, stored in sold_gst[]
* 10. sale() - recursion hata ke while loop lagaya (no stack overflow)
* 11. scanf("%s", &pass) - & hata diya (char array mein & wrong tha)
* 12. rate[pos-1],stockupdate useless expression fix kiya
* 13. Password set hone par confirmation print hoti hai
* 14. Input length limits added (buffer overflow protection)
* 15. itemlistresult() - local loop variable use karta hai ab
* 16. Billing history mein correct per-item GST show hoti hai
* 17. sold_price[] sirf base amount store karta hai, GST alag track hoti hai
*/

#include <stdio.h>

#include <string.h>

```

```

#include <ctype.h>

#include <stdlib.h>


/* —— Constants —— */

#define MAX_PRODUCTS 10000

#define MAX_SALES 50000

#define MAX_ATTEMPTS 3

#define NAME_LEN 50

#define PASS_LEN 50

#define GST_PERCENT 2


/* —— Global Data —— */

int n = 0;          /* number of products entered */

char product[MAX_PRODUCTS][NAME_LEN];

int rate[MAX_PRODUCTS];

int num[MAX_PRODUCTS];    /* stock quantity */


/* Sales history */

char sold_product[MAX_SALES][NAME_LEN];

int sold_qty[MAX_SALES];

int sold_price[MAX_SALES];    /* base price (before GST) */

int sold_gst[MAX_SALES];    /* GST amount per sale entry */

int sale_count = 0;

int total_bill = 0;    /* running total of current session */


/* Department passwords */

char pass1[PASS_LEN] = ""; /* product dept - anjali */

char pass2[PASS_LEN] = ""; /* pricing dept - mili */

```

```

char pass3[PASS_LEN] = ""; /* stock dept - ritu */
char pass4[PASS_LEN] = ""; /* maintain dept - manager */
char pass5[PASS_LEN] = ""; /* sales dept - salesman */
char pass6[PASS_LEN] = ""; /* owner/admin - owner */


/* —— Forward Declarations —— */

void password();
void menu();
void item();
void price();
void stock();
void maintain();
void salesteam();
void sale();
void admin_system();
void itemlistresult();
int validate_password(const char *p);
int login(const char *expected_id, const char *stored_pass, const char *dept_name);
void print_separator(const char *title);
void safe_input(char *buf, int size);
void show_menu_prompt();


void show_menu_prompt() {
    int choicce, a;

    printf("enter 1 for show menu press 0 for exit");

    scanf("%d",&choicce);

    if(choicce == 1){
        a=1;
    }
}

```

```

        }

    else

    {

        a=0;

        printf("Thank you for visiting");

    }

}

```

```

/* =====

UTILITY: Print a neat separator with title

===== */

void print_separator(const char *title) {

    printf("\n");

    printf("=====\\n");

    if (title && title[0])

        printf(" %s\\n", title);

    printf("=====\\n");

}

```

```

/* =====

UTILITY: Safe string input (prevents buffer overflow)

===== */

void safe_input(char *buf, int size) {

    if (fgets(buf, size, stdin)) {

        buf[strcspn(buf, "\\n")] = '\\0'; /* remove trailing newline */

    }

}

```

```

/* =====

PASSWORD VALIDATOR

Rules: min 6 chars, at least 1 letter, 1 digit, 1 symbol

Returns 1 if valid, 0 if invalid

===== */

int validate_password(const char *p) {
    int len = 0, has_letter = 0, has_digit = 0, has_symbol = 0;
    if (!p) return 0;
    while (p[len]) {
        if (isalpha((unsigned char)p[len])) {
            has_letter = 1;
        }
        else if (isdigit((unsigned char)p[len]))
        {
            has_digit = 1;
        }

        else{
            has_symbol = 1;
        }

        len++;
    }
    if (len < 6) {
        printf(" Password too short. Minimum 6 characters required.\n");
        return 0;
    }
    if (!has_letter) { printf(" Password must contain at least one letter.\n"); return 0; }
}

```

```

    if (!has_digit) { printf(" Password must contain at least one digit.\n"); return 0; }

    if (!has_symbol) { printf(" Password must contain at least one symbol (e.g. @$!).\n");
return 0; }

    return 1;
}

```

```

/* =====

```

```

LOGIN HELPER

```

```

Returns 1 on success, 0 on failure (after MAX_ATTEMPTS)

```

```

===== */

```

```

int login(const char *expected_id, const char *stored_pass, const char *dept_name) {
    char entered_id[NAME_LEN], entered_pass[PASS_LEN];
    int attempts;

```

```

    /* ---- ID check ---- */

```

```

    printf("Enter User ID for %s: ", dept_name);

```

```

    scanf(" ");

```

```

    safe_input(entered_id, NAME_LEN);

```

```

    if (strcmp(entered_id, expected_id) != 0) {

```

```

        printf(" Invalid User ID. Access Denied.\n");

```

```

        return 0;

```

```

    }

```

```

    /* ---- Password check with max 3 attempts ---- */

```

```

    for (attempts = 1; attempts <= MAX_ATTEMPTS; attempts++) {

```

```

        printf("Enter Password (%d/%d): ", attempts, MAX_ATTEMPTS);

```

```

        scanf(" ");

```

```

safe_input(entered_pass, PASS_LEN);

if (strcmp(entered_pass, stored_pass) == 0) {
    printf("Login Successful. Welcome, %s!\n\n", expected_id);
    return 1;
} else {
    if (attempts < MAX_ATTEMPTS)
        printf("Wrong password. %d attempt(s) remaining.\n", MAX_ATTEMPTS -
attempts);
    else
        printf("Maximum attempts reached. Access Blocked. Try again later.\n");
    }
}
return 0;
}

/* =====

SET PASSWORDS (called once at startup)

===== */

void password() {
    char buf[PASS_LEN];

    print_separator("DEPARTMENT PASSWORD SETUP");
    printf("NOTE: Password must have at least 6 characters,\n");
    printf("    1 letter, 1 digit, and 1 symbol (e.g. @$!)\n\n");

    /* -- Product Dept -- */

    do {

```

```

    printf("Set password for PRODUCT DEPARTMENT (User: anjali): ");

    scanf(" "); safe_input(buf, PASS_LEN);
} while (!validate_password(buf));

strcpy(pass1, buf);

printf("Product dept password set.\n\n");


/* -- Pricing Dept -- */
do {
    printf("Set password for PRICING DEPARTMENT (User: mili): ");

    scanf(" "); safe_input(buf, PASS_LEN);
} while (!validate_password(buf));

strcpy(pass2, buf);

printf(" Pricing dept password set.\n\n");


/* -- Stock Dept -- */
do {
    printf("Set password for STOCK DEPARTMENT (User: ritu): ");

    scanf(" "); safe_input(buf, PASS_LEN);
} while (!validate_password(buf));

strcpy(pass3, buf);

printf(" Stock dept password set.\n\n");


/* -- Maintain Dept -- */
do {
    printf("Set password for MAINTENANCE DEPARTMENT (User: manager): ");

    scanf(" "); safe_input(buf, PASS_LEN);
} while (!validate_password(buf));

strcpy(pass4, buf);

```



```

printf(" Maintenance dept password set.\n\n");

/* -- Sales Dept -- */
do {
    printf("Set password for SALES DEPARTMENT (User: salesman): ");
    scanf(" "); safe_input(buf, PASS_LEN);
} while (!validate_password(buf));
strcpy(pass5, buf);
printf(" Sales dept password set.\n\n");

/* -- Owner/Admin -- */
do {
    printf("Set password for OWNER / ADMIN (User: owner): ");
    scanf(" "); safe_input(buf, PASS_LEN);
} while (!validate_password(buf));
strcpy(pass6, buf);
printf(" Admin password set.\n\n");

print_separator("ALL PASSWORDS SET SUCCESSFULLY");
}

/* =====

MAIN MENU

===== */

void menu() {
    int choice, choicce, a=1;

```

```

while (a) {

    print_separator("MAIN MENU");

    printf(" 1. Add Products\n");
    printf(" 2. Add / Update Price\n");
    printf(" 3. Add / Update Stock\n");
    printf(" 4. Maintain / Update Records\n");
    printf(" 5. Sale Products\n");
    printf(" 6. View Reports (Admin)\n");
    printf(" 7. Exit\n");

    printf("-----\n");

    printf("Enter your choice: ");

    if (scanf("%d", &choice) != 1) {

        printf(" Invalid input. Please enter a number.\n");

        while (getchar() != '\n'); /* flush buffer */

        continue;
    }

    switch (choice) {

        case 1:

            item();

            show_menu_prompt();

            break;

        case 2:

            if (n == 0) { printf(" No products added yet. Please add products first.\n"); break; }

            price();

            show_menu_prompt();

```

```
        break;
    case 3:
        if (n == 0) { printf(" No products added yet. Please add products first.\n"); break; }
        stock();
        show_menu_prompt();
        break;
    case 4:
        if (n == 0) { printf(" No records to maintain. Please add products first.\n"); break; }
        maintain();
        show_menu_prompt();
        break;
    case 5:
        if (n == 0) { printf(" No products available. Please set up inventory first.\n");
break; }
        salesteam();
        show_menu_prompt();
        break;
    case 6:
        admin_system();
        show_menu_prompt();
        break;
    case 7:
        print_separator("THANK YOU FOR USING THE SYSTEM");
        return;
    default:
        printf(" Invalid choice. Please enter 1-7.\n");
    }
}
```

```
}
```

```
/* =====  
SHOW PRODUCT LIST (with price & stock if available)  
===== */
```

```
void itemlistresult() {  
    int j;  
    printf("\n--- Current Product List ---\n");  
    for (j = 0; j < n; j++) {  
        printf(" %2d. %-20s Price: Rs. %-6d Stock: %d\n",  
            j + 1, product[j], rate[j], num[j]);  
    }  
    printf("-----\n");  
}
```

```
/* =====  
ADD PRODUCTS (Dept: anjali / pass1)  
===== */
```

```
void item() {  
    int i, count;  
  
    print_separator("PRODUCT DEPARTMENT - ADD PRODUCTS");  
  
    if (!login("anjali", pass1, "Product Department"))  
        return;
```

```

printf("Enter number of items to add (1-100): ");
if (scanf("%d", &count) != 1 || count <= 0 || count > MAX_PRODUCTS) {
    printf(" Invalid number. Must be between 1 and %d.\n", MAX_PRODUCTS);
    while (getchar() != '\n');
    return;
}

if (n + count > MAX_PRODUCTS) {
    printf(" Cannot add %d items. Only %d slots remaining.\n", count, MAX_PRODUCTS
- n);
    return;
}

printf("Enter product names:\n");
for (i = n; i < n + count; i++) {
    printf(" Product %d: ", i + 1);
    scanf(" "); safe_input(product[i], NAME_LEN);
    rate[i] = 0;
    num[i] = 0;
}
n += count;

itemlistresult();

printf(" %d product(s) added successfully. Total products: %d\n", count, n);
}

/* =====
SET / UPDATE PRICE (Dept: mili / pass2)

```

```
=====*/
```

```
void price() {  
    int i, r;  
  
    print_separator("PRICING DEPARTMENT - SET PRICES");  
  
    if (!login("mili", pass2, "Pricing Department"))  
        return;  
  
    itemlistresult();  
  
    printf("Enter prices for all products:\n");  
    for (i = 0; i < n; i++) {  
  
        if (rate[i] != 0)  
            continue; // skip products whose price is already set  
  
        do {  
            printf(" Price of %-20s: Rs. ", product[i]);  
  
            if (scanf("%d", &r) != 1 || r <= 0) {  
                printf(" Invalid price. Must be a positive number.\n");  
                while (getchar() != '\n');  
                r = -1;  
            }  
  
        } while (r <= 0);  
    }  
}
```

```

    rate[i] = r;
}

print_separator("UPDATED PRICE LIST");
itemlistresult();
printf(" Prices updated successfully.\n");
}

/* =====
ADD / UPDATE STOCK (Dept: ritu / pass3)
===== */
void stock() {
    int i, s;

    print_separator("STOCK DEPARTMENT - UPDATE STOCK");

    if (!login("ritu", pass3, "Stock Department"))
        return;

    itemlistresult();

    printf("Enter stock quantity for all products:\n");
    for (i = 0; i < n; i++) {
        if(num[i] != 0)
            continue;
        do {
            printf(" Stock of %-20s: ", product[i]);
            if (scanf("%d", &s) != 1 || s < 0) {

```

```

        printf(" Invalid quantity. Must be 0 or more.\n");

        while (getchar() != '\n');

        s = -1;

    }

}

while (s < 0);

num[i] = s;

}

print_separator("UPDATED STOCK LIST");

itemlistresult();

printf(" Stock updated successfully.\n");

}

/* =====

MAINTAIN / UPDATE RECORDS (Dept: manager / pass4)

===== */

void maintain() {

    int count;

    int choice, pos, new_rate, new_stock;

    char new_name[NAME_LEN];

    print_separator("MAINTENANCE DEPARTMENT - UPDATE RECORDS");

    if (!login("manager", pass4, "Maintenance Department"))

        return;

    while (1) {

```



```

itemlistresult();

printf("\nWhat do you want to update?\n");

printf(" 1. Update Product Name\n");

printf(" 2. Update Price\n");

printf(" 3. Update Stock\n");

printf(" 4. Update All (Name + Price + Stock)\n");

printf(" 5. Back to Main Menu\n");

printf("Choice: ");

if (scanf("%d", &choice) != 1) { while (getchar() != '\n'); continue; }

if (choice == 5) break;

if (choice < 1 || choice > 5) {
    printf(" Invalid choice.\n");
    continue;
}

printf("Enter product position (1-%d): ", n);

if (scanf("%d", &pos) != 1 || pos < 1 || pos > n) {
    printf(" Invalid position.\n");
    while (getchar() != '\n');
    continue;
}

if (choice == 1 || choice == 4) {
    printf("Enter new product name: ");
    scanf(" "); safe_input(new_name, NAME_LEN);
    strcpy(product[pos - 1], new_name);
}

```

```

    printf("Product name updated.\n");
}
if (choice == 2 || choice == 4) {
    do {
        printf("Enter new price for %s: Rs. ", product[pos - 1]);
        if (scanf("%d", &new_rate) != 1 || new_rate <= 0) {
            printf(" Invalid price.\n");
            while (getchar() != '\n');
            new_rate = -1;
        }
    } while (new_rate <= 0);
    rate[pos - 1] = new_rate;
    printf("Price updated.\n");
}
if (choice == 3 || choice == 4) {
    do {
        printf("Enter new stock for %s: ", product[pos - 1]);
        if (scanf("%d", &new_stock) != 1 || new_stock < 0) {
            printf(" Invalid stock.\n");
            while (getchar() != '\n');
            new_stock = -1;
        }
    } while (new_stock < 0);
    num[pos - 1] = new_stock;
    printf(" Stock updated.\n");
}

print_separator("UPDATED RECORDS");

```

```

        itemlistresult();

        printf("\nDo you still want to update something?\n");
printf(" 1. Yes (continue)\n");
printf(" 2. No (exit)\n");
printf("Choice: ");

if (scanf("%d", &count) != 1) {
    while (getchar() != '\n');
    count = 2;
}

if (count == 2) {
    printf("Exiting Maintenance Module...\n");
    break;
}

}

/* =====

SALES TEAM VIEW (Dept: salesman / pass5)

===== */

void salesteam() {
    print_separator("SALES DEPARTMENT - PRODUCT VIEW");

    if (!login("salesman", pass5, "Sales Department"))
        return;

    itemlistresult();

```

```

printf("Product list loaded. Proceeding to sale...\n");

sale();
}

/* =====

PROCESS SALE

=====*/

void sale() {
    int pos, number, bill_amount, gst_amount, choice;

    total_bill = 0; /* reset for this sales session */

    while (1) {
        print_separator("BILLING - SALE PRODUCT");
        itemlistresult();

        printf("Enter position of product to sell (1-%d): ", n);
        if (scanf("%d", &pos) != 1 || pos < 1 || pos > n) {
            printf("Invalid position.\n");
            while (getchar() != '\n');
            continue;
        }

        if (num[pos - 1] == 0) {
            printf(" Product '%s' is OUT OF STOCK.\n", product[pos - 1]);
        } else {
            printf("Enter quantity to sell (Available: %d): ", num[pos - 1]);

```

```

if (scanf("%d", &number) != 1 || number <= 0) {
    printf(" Invalid quantity.\n");
    while (getchar() != '\n');
    continue;
}

if (number > num[pos - 1]) {
    printf(" Insufficient stock! Available: %d\n", num[pos - 1]);
} else {
    /* ---- Process the sale ---- */

    bill_amount = number * rate[pos - 1];
    gst_amount = (bill_amount * GST_PERCENT) / 100;
    total_bill += bill_amount + gst_amount;

    /* Store in history */
    if (sale_count < MAX_SALES) {
        strcpy(sold_product[sale_count], product[pos - 1]);
        sold_qty[sale_count] = number;
        sold_price[sale_count] = bill_amount;
        sold_gst[sale_count] = gst_amount;
        sale_count++;
    }

    num[pos - 1] -= number;

    /* ---- Print Receipt ---- */
    print_separator("SALE RECEIPT");
    printf(" Product    : %s\n", product[pos - 1]);
    printf(" Unit Price  : Rs. %d\n", rate[pos - 1]);

```

```

        printf(" Quantity   : %d\n", number);

        printf(" Base Amount : Rs. %d\n", bill_amount);

        printf(" GST (%d%%)   : Rs. %d\n", GST_PERCENT, gst_amount);

        printf(" Item Total  : Rs. %d\n", bill_amount + gst_amount);

        printf(" Remaining Stk: %d\n", num[pos - 1]);

        printf(" SESSION TOTAL: Rs. %d\n", total_bill);

        printf("=====\n");
    }
}

printf("\nSell another product? (1=Yes / 2=No): ");
if (scanf("%d", &choice) != 1) { while (getchar() != '\n'); choice = 2; }
if (choice != 1) {
    printf("\n Session ended. Total billed this session: Rs. %d\n", total_bill);
    break;
}
}
}

/* =====

ADMIN / OWNER REPORT (owner / pass6)

===== */

void admin_system() {
    int i, grand_total = 0;

    print_separator("ADMIN PANEL - FULL REPORT");

    if (!login("owner", pass6, "Admin / Owner"))

```

```

return;

/* ---- Inventory Report ---- */

print_separator("INVENTORY STATUS");

if (n == 0) {
    printf(" [No products in inventory]\n");
} else {
    printf(" %-4s %-22s %-12s %-10s %-15s\n",
        "No.", "Product", "Price(Rs.)", "Stock", "Stock Value(Rs.)");
    printf(" %-4s %-22s %-12s %-10s %-15s\n",
        "---", "-----", "-----", "----", "-----");
    for (i = 0; i < n; i++) {
        printf(" %-4d %-22s %-12d %-10d %-15d\n",
            i + 1, product[i], rate[i], num[i], rate[i] * num[i]);
    }
}

/* ---- Sales / Billing Report ---- */

print_separator("BILLING HISTORY");

if (sale_count == 0) {
    printf(" [No sales recorded yet]\n");
} else {
    printf(" %-4s %-22s %-8s %-12s %-10s %-12s\n",
        "No.", "Product", "Qty", "Base(Rs.)", "GST(Rs.)", "Total(Rs.)");
    printf(" %-4s %-22s %-8s %-12s %-10s %-12s\n",
        "---", "-----", "---", "-----", "-----", "-----");
    for (i = 0; i < sale_count; i++) {
        int item_total = sold_price[i] + sold_gst[i];
    }
}

```

```

        grand_total += item_total;

        printf(" %-4d %-22s %-8d %-12d %-10d %-12d\n",
               i + 1, sold_product[i], sold_qty[i],
               sold_price[i], sold_gst[i], item_total);
    }

    printf("\n Grand Total (incl. GST): Rs. %d\n", grand_total);
}

print_separator("END OF REPORT");
}

/* =====

MAIN ENTRY POINT

===== */

int main() {
    int choice;

    print_separator("INVENTORY MANAGEMENT SYSTEM");
    printf(" Multi-Department | GST Enabled | Role-Based Access\n");
    print_separator(NULL);

    printf("Before using the system, you must set passwords\n");
    printf("for each department.\n\n");
    printf(" 1. Set Passwords and Start\n");
    printf(" 2. Exit\n");
    printf("Choice: ");

    if (scanf("%d", &choice) != 1) {

```



```
    printf(" Invalid input. Exiting.\n");  
    return 1;  
}  
  
if (choice == 1) {  
    password();  
    menu();  
} else if (choice == 2) {  
    print_separator("THANK YOU FOR VISITING");  
} else {  
    printf(" Invalid input. Exiting.\n");  
}  
  
return 0;  
}
```